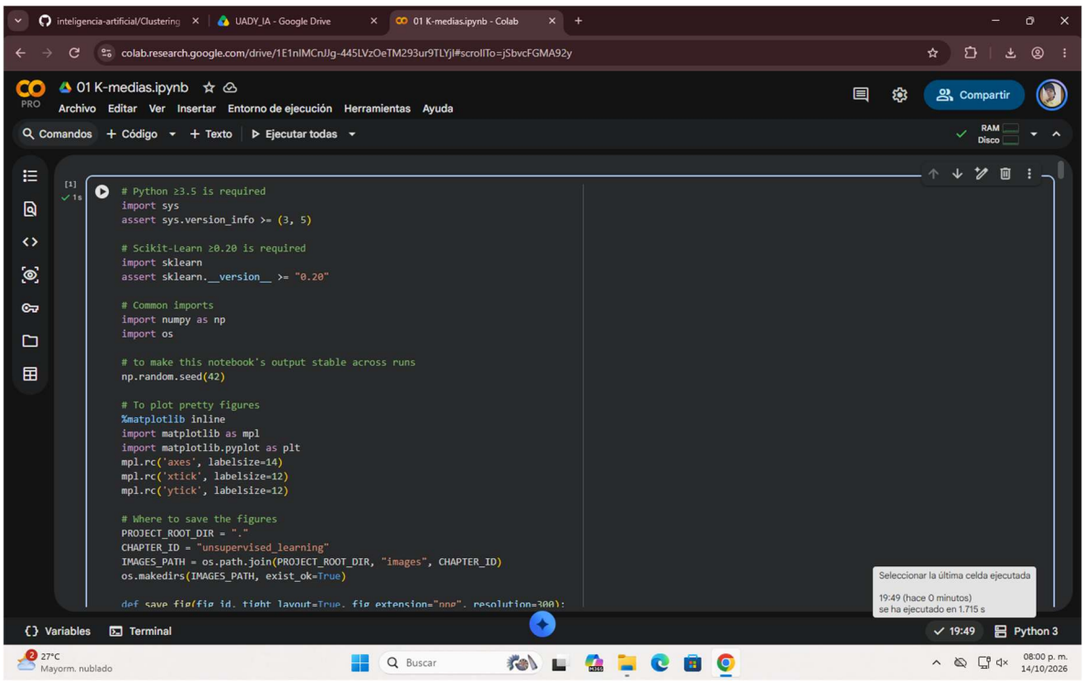
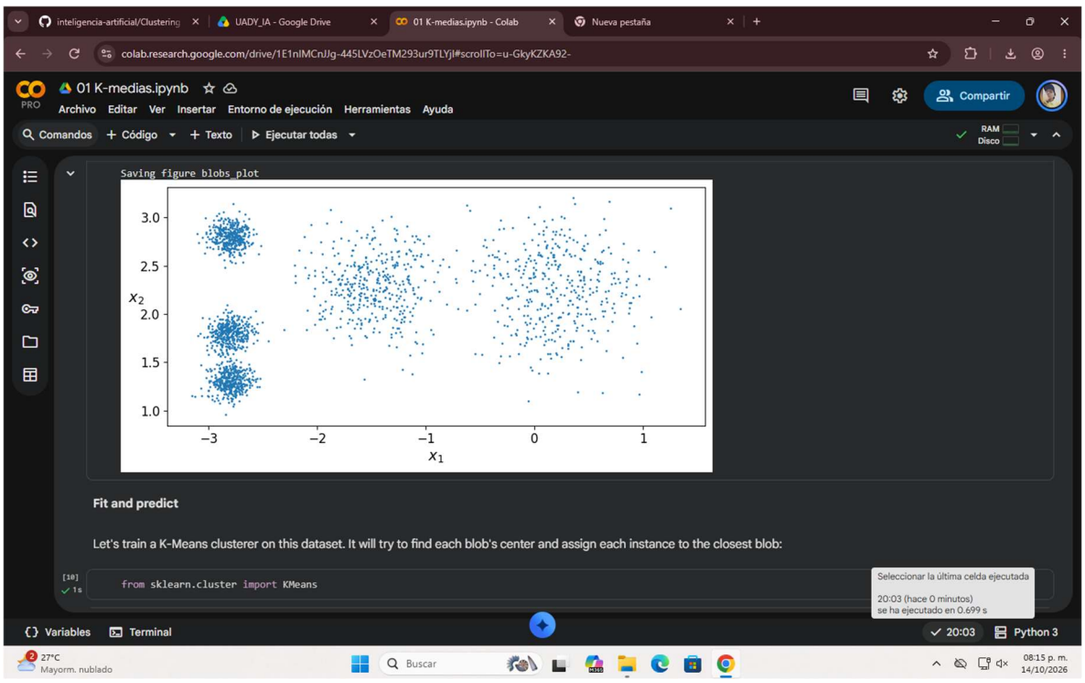
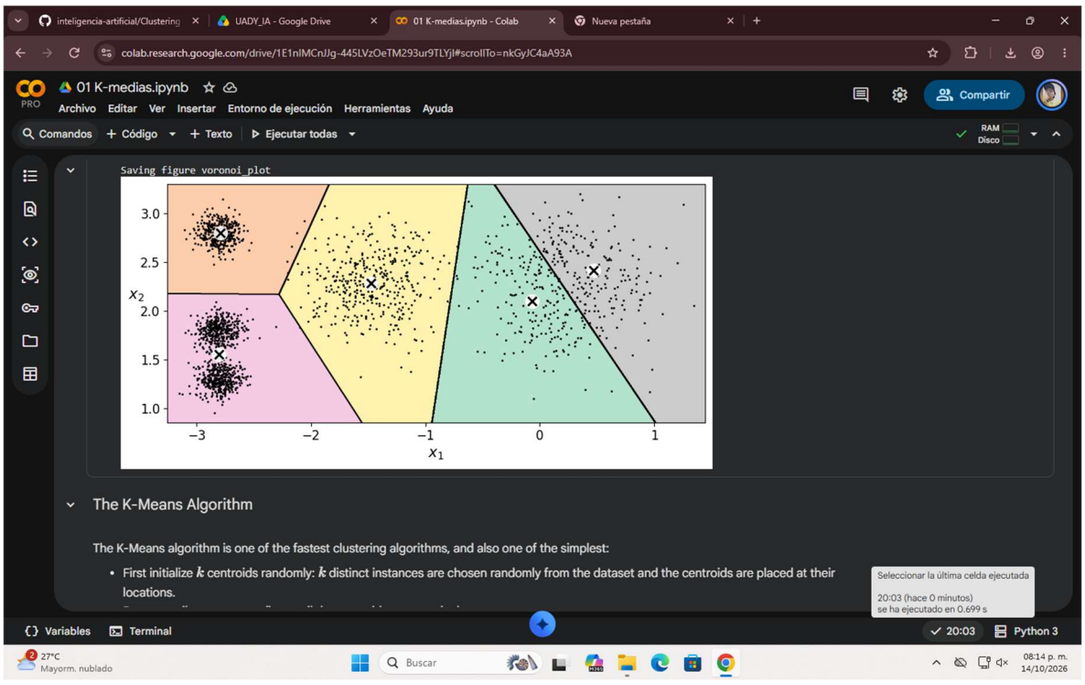


CAPTURAS PRIMERA CORRIDA

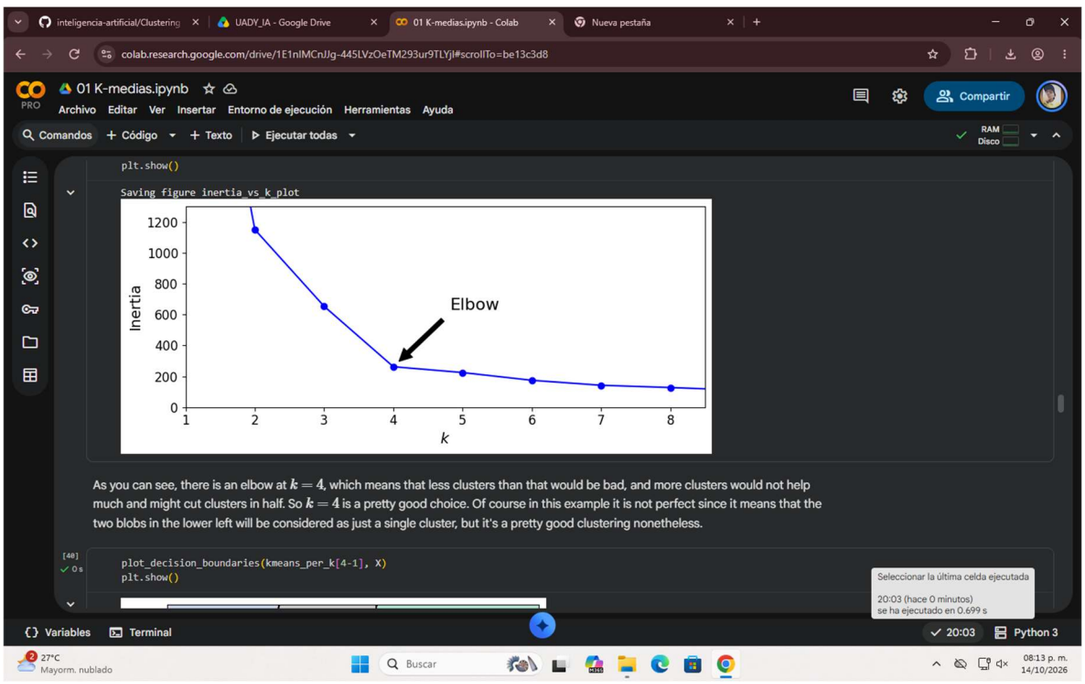




The K-Means Algorithm

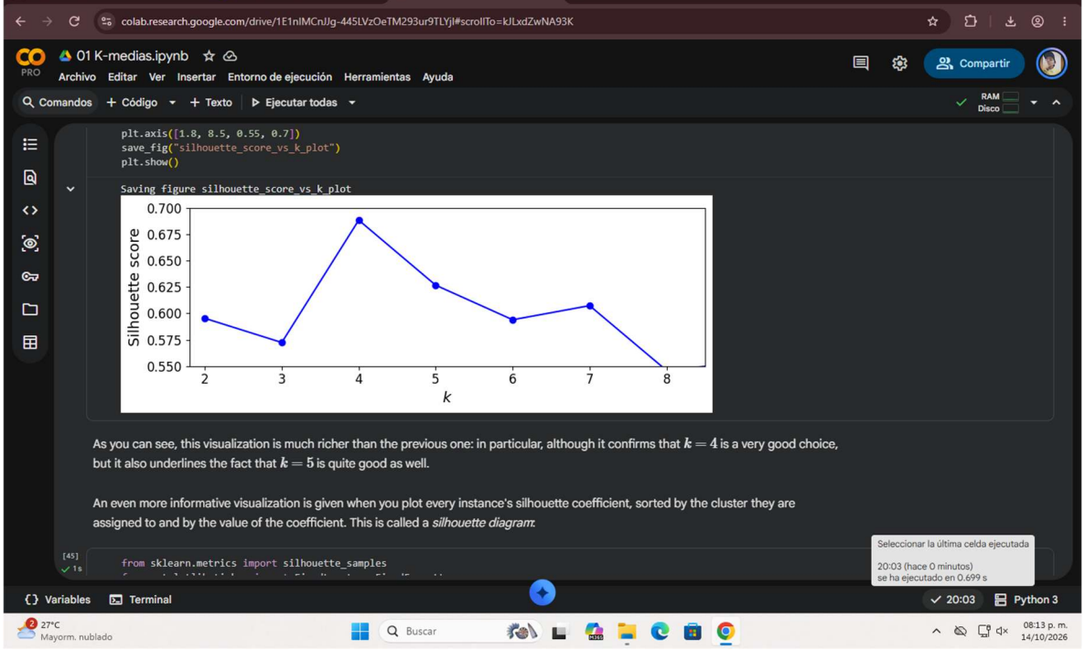
The K-Means algorithm is one of the fastest clustering algorithms, and also one of the simplest:

- First initialize k centroids randomly: k distinct instances are chosen randomly from the dataset and the centroids are placed at their locations.



As you can see, there is an elbow at $k = 4$, which means that less clusters than that would be bad, and more clusters would not help much and might cut clusters in half. So $k = 4$ is a pretty good choice. Of course in this example it is not perfect since it means that the two blobs in the lower left will be considered as just a single cluster, but it's a pretty good clustering nonetheless.

```
[40]
plot_decision_boundaries(kmeans_per_k[4-1], X)
plt.show()
```

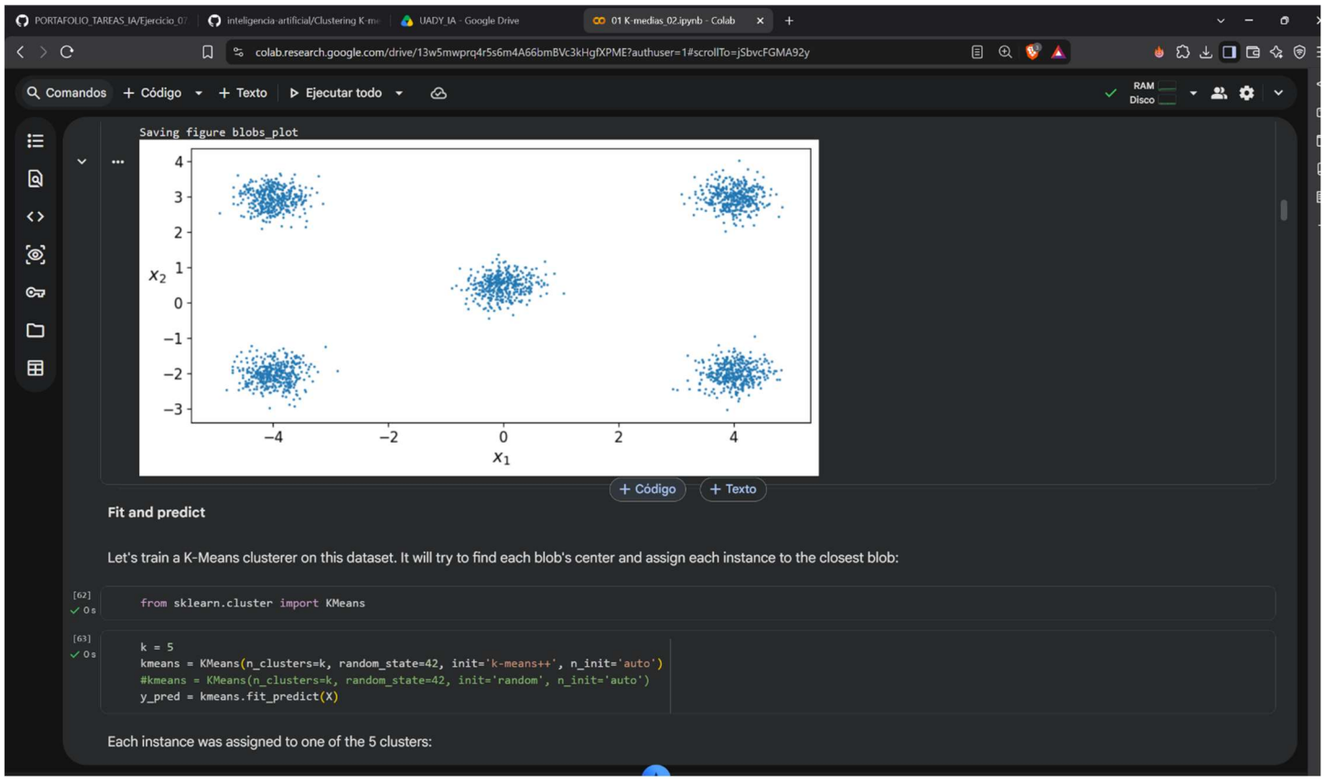


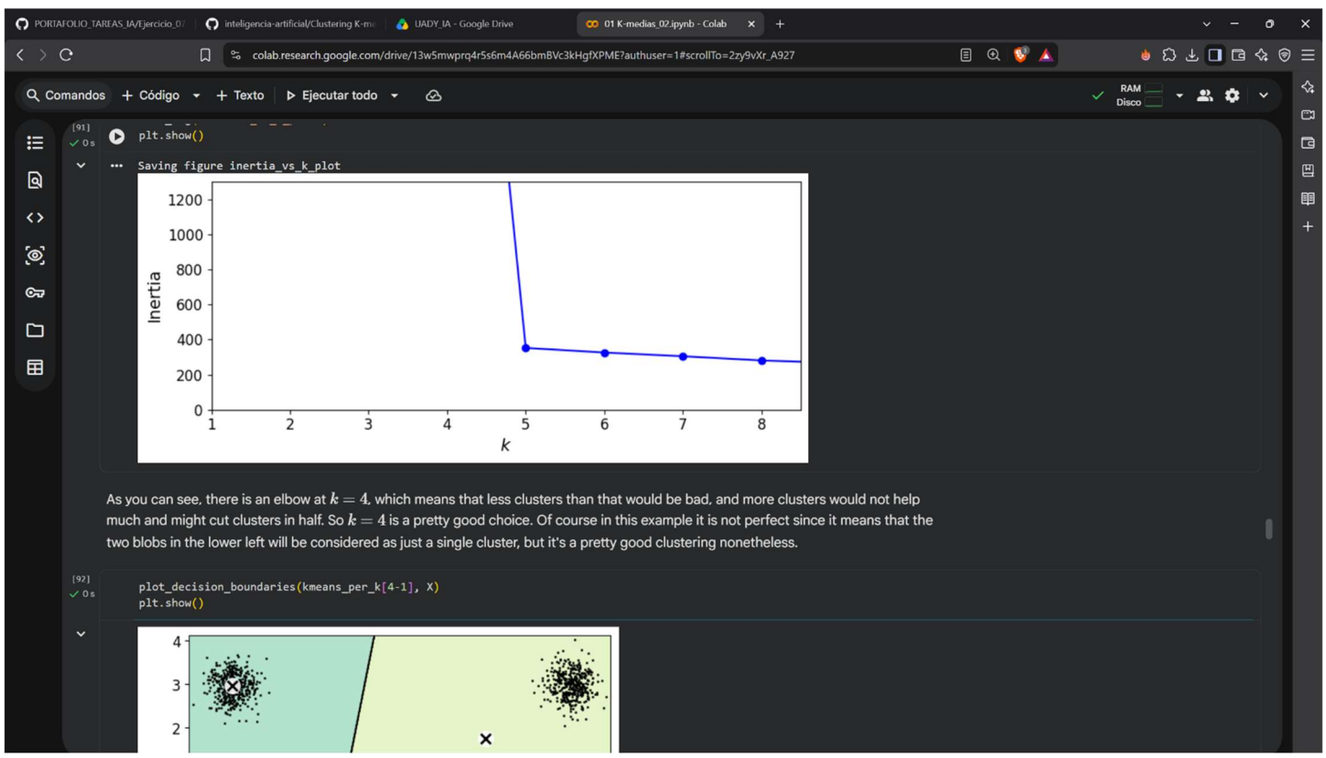
As you can see, this visualization is much richer than the previous one: in particular, although it confirms that $k = 4$ is a very good choice, but it also underlines the fact that $k = 5$ is quite good as well.

An even more informative visualization is given when you plot every instance's silhouette coefficient, sorted by the cluster they are assigned to and by the value of the coefficient. This is called a *silhouette diagram*:

```
[45]
from sklearn.metrics import silhouette_samples
```

CAPTURAS SEGUNDA CORRIDA



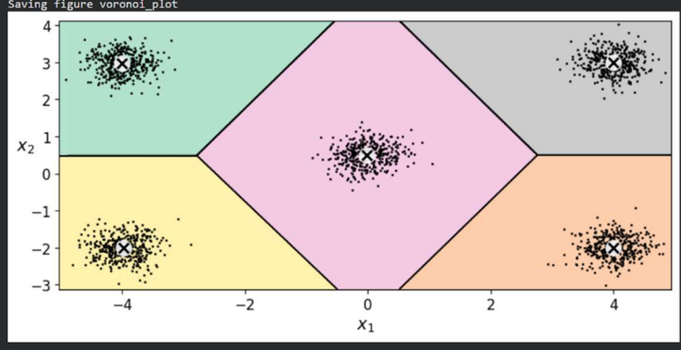


PORTAFOLIO_TAREAS_IA/Ejercicio_07 | inteligencia-artificial/Clustering K-m | UADY_IA - Google Drive | 01 K-medias_02.ipynb - Colab

colab.research.google.com/drive/13w5mwprq4r5s6m4A66bmBVc3kHgfXPME?authuser=1#scrollTo=2zy9vXr_A927

Comandos | + Código | + Texto | Ejecutar todo | RAM | Disco

```
Saving figure voronoi_plot
```



The K-Means Algorithm

The K-Means algorithm is one of the fastest clustering algorithms, and also one of the simplest:

- First initialize k centroids randomly: k distinct instances are chosen randomly from the dataset and the centroids are placed at their locations.
- Repeat until convergence (i.e., until the centroids stop moving):
  - Assign each instance to the closest centroid.
  - Update the centroids to be the mean of the instances that are assigned to them.

The `KMeans` class applies an optimized algorithm by default. To get the original K-Means algorithm (for educational purposes only), you

PORTAFOLIO_TAREAS_IA/Ejercicio_07 | inteligencia-artificial/Clustering K-m | UADY_IA - Google Drive | 01 K-medias_02.ipynb - Colab

colab.research.google.com/drive/13w5mwprq4r5s6m4A66bmBVc3kHgfXPME?authuser=1#scrollTo=rTfalG_fA93L

Comandos | + Código | + Texto | Ejecutar todo | RAM | Disco

Let's plot the silhouette score as a function of k:

[93] ✓ 0 s
```
from sklearn.metrics import silhouette_score
```

[94] ✓ 0 s
```
silhouette_score(X, kmeans.labels_)
```
```
np.float64(0.5847221512767647)
```

[95] ✓ 0 s
```
silhouette_scores = [silhouette_score(X, model.labels_)
                     for model in kmeans_per_k[1:]]
```

[96] ✓ 0 s
```
plt.figure(figsize=(8, 3))
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Silhouette score", fontsize=14)
plt.axis([1.8, 8.5, 0.55, 0.7])
save_fig("silhouette_score_vs_k_plot")
plt.show()
```
```
Saving figure silhouette_score_vs_k_plot
```

